

Master's Programme in Electrical and Automation Engineering

Automated web-based testing framework for the interface of a power plant control

Duong Truong

Master's thesis
2025

Copyright ©2025 Duong Truong

Author Duong Truong		
Title of thesis Automated web-based testing framework for the interface of a power plant control		
Programme Electrical and Automation Engineering		
Major Name Control, Robotics and Autonomous Systems		
Thesis supervisor Belahcen Anouar		
Thesis advisor(s) Jonatan Rösgren		
Date 30.04.2025	Number of pages 54	Language English

Abstract

Human–Machine Interfaces (HMIs) in factory control rooms demand the highest levels of reliability and safety. Traditional manual testing of web-based HMIs is labour-intensive, prone to human mistake, and unable to keep pace with modern agile workflows. This thesis presents the design, implementation, and evaluation of an end-to-end automated testing framework for web-based HMI. Leveraging Playwright for reliable browser automation, Docker for environment consistency, and Bamboo/Bitbucket for continuous integration and continuous development, the framework embeds automated tests directly into the pull-request workflow to provide immediate feedback on every commit.

A comparative analysis of leading test frameworks (Selenium, Cypress, Puppeteer, Nightwatch, Playwright) against project-specific criteria, demonstrated Playwright’s superiority in minimizing flakiness and maximizing maintainability. The implemented solution executes modular end-to-end scenarios within containerized environment, generating interactive reports. Over 45 commit-triggered runs, the mean execution time was 275 seconds with a flakiness rate of 4.4 % (eliminated by a single retry), and 100 % pull-request gating effectiveness.

By benchmarking against ISO 9241-210 and ANSI/ISA-101.01-2015, this work establishes a human and safety centred testing baseline. The framework replaces manual regression testing with a reproducible, rapid feedback mechanism, significantly reducing risk and accelerating development velocity.

Keywords Automated testing; end-to-end (E2E); Human–Machine Interface; Playwright; CI/CD; Docker; safety-critical systems; HMI testing.

Table of contents

1	Introduction.....	6
2	Literature review.....	9
2.1	Software testing	9
2.1.1	Automated testing	9
2.1.2	Criteria for Automated Testing	10
2.1.3	End-to-end testing.....	11
2.1.4	Best practices for E2E testing	11
2.1.5	Test case design	12
2.1.6	Human–Machine Interfaces in Safety-Critical Systems.....	13
2.2	Continuous Integration and Continuous Deployment in Automated Testing.....	14
2.3	Data Generation and Reporting in Automated Testing	15
2.4	Comparative Analysis of Manual vs. Automated Testing Effort and Effectiveness.....	15
3	Case Study	17
3.1	Motivation	17
3.2	System Context	17
3.3	Objectives and limitations	18
3.4	Stakeholder consultations.....	18
4	Framework Design.....	20
4.1	Overall Framework Architecture	20
4.2	Tool Selection and Comparative Analysis	21
4.2.1	Tool selection criteria	21
4.2.2	Comparison	23
4.3	Framework architecture	25
4.3.1	Playwright	27
4.3.2	Continuous Integration and Continuous Deployment	28
4.3.3	Docker image.....	30
4.3.4	Report.....	31
4.4	Application of International Standards.....	32
4.4.1	Standard ISO	33
4.4.2	ANSI/ISA-101.01-2015	35

4.4.3	Best practice in E2E testing	36
4.4.4	Current alignment with Standard and best practice	37
4.5	Summary	39
5	Implementation	41
5.1	Server	41
5.2	Implementation	41
6	Discussion.....	44
7	Conclusion	46
8	Future work	47
	References.....	49
	Appendix.....	52

1 Introduction

Over the past decade, testing has received increasing attention for Human–Machine Interfaces (HMIs) used in industrial manufacturing, where a single millisecond lag can endanger human lives and result in unplanned downtime costing millions of dollars (Ruari, n.d.) (Wang, et al., 2020). Therefore, ensuring the performance reliability as well as usability of HMIs is a top priority. HMI testing is typically manually conducted, in which testers simulate operator interactions such as clicks, data entries, navigation flows, and verify that the interface responds correctly by checking graphical layouts and control commands. On the other hand, automated frameworks generally integrate with continuous integration (CI) and continuous delivery (CD) pipelines to run these tests on every software update, ensuring that reliability is maintained without manual effort. Poor interface design combined with insufficient testing has been shown to significantly contribute to human error in industrial settings. For example, a series of operator mistakes at Chernobyl were traced back to inadequate system feedback and design shortcomings (Wang, et al., 2020).

Research has revealed that manual testing methods have struggled to provide the comprehensive coverage and fast feedback loop which are required by modern agile and DevOps development lifecycles (Beata & Agnieszka, 2024). Moreover, researchers have found that manual testing has been time-consuming, error-prone and has often failed to scale with the growing complexity of modern web-based HMIs (Himanshu, n.d.). Although extensive work has advanced automated user interface (UI) testing for general web applications, no dedicated framework has been designed for industrial adoption in the high-throughput, safety-critical HMI domain (Iva, et al., 2025). Furthermore, recent evaluations have compared the of leading capabilities automated testing tools but have not suggested on a single, unified framework or provided definitive guidance adapted to industrial HMI environments.

Recently, several comparative studies have evaluated automated UI tools for industrial HMIs, such as Selenium and Playwright. These studies report that while Selenium’s mature ecosystem provides broad browser support, its synchronization challenges make it less reliable for dynamic interfaces (Beata & Agnieszka, 2024). On the other hand, Playwright’s native auto-waiting and cross-browser contexts are promising for more reliable HMI test execution, along with its easy integration into CI/CD pipelines. Furthermore, recent case studies have demonstrated that these advanced frameworks can capture both technical performance metrics and usability indicators under realistic operational conditions, closing gaps left by manual methods (Saad, et al., 2025) (Beata & Agnieszka, 2024)

One promising solution for addressing the lack of specialized testing frameworks in industrial HMI environments would be to embed an end-to-end automated test pipeline into CI/CD workflow - architecture design benchmarked against industrial standards - so that every interface update automatically triggers graphical, functional, and performance checks. This solution leverages the built-in synchronization and multi-browser support of modern end-to-end frameworks to minimize flakiness on dynamic industrial HMIs, while its continuous CI/CD integration offers the rapid feedback which is essential to agile and DevOps practices. Furthermore, by evaluating the framework against industrial standards ISO 9241-210:2010 and ANSI/ISA-101.01- 2015, this evaluation ensures the technical robustness and compliance of the framework with established safety benchmarks in industrial factory.

Therefore, the aim of this thesis is to design, implement and evaluate an end-to-end automated testing framework embedding Playwright tests into the CI/CD pipelines. The framework will replace the current manual testing, providing repeatable feedback on functional correctness and real-time performance at every HMI update regardless of the size of the change. To realise this aim, the thesis will identify the most suitable architectural approach for automated testing, accomplished by conducting comparative analysis of different testing frameworks, focusing on their capabilities for CI/CD integration as well as evaluating their cost-effectiveness and long-term maintainability (Rudolf, et al., 2014). The framework in this thesis focuses on simulating typical operator interaction flows to ensure the test case covers the most critical features in operating a power plant. Additionally, this thesis also studies and discusses the potential integration of international standards and best practices to ensure that the testing framework meets the strict industry benchmarks. The industrial standards ISO 9241-210:2010 and ANSI/ISA-101.01-2015 are used as guidelines of critical requirements to inform the design and evaluation of HMIs in automation systems (ISO, 2019) (Automation, 2015)

The scope of this thesis is limited to the implementation and evaluation of end-to-end automated testing framework for web-based HMIs used in power-plant control rooms. It addresses the front-end interface layer and assumes that the underlying control-system APIs and back-end services are already functionally correct. The work focuses on simulated user-flow scenarios rather than live operator trials and on continuous integration within an existing CI/CD pipeline. Unit testing and hardware validation are not considered in this thesis. The comparative tool analysis is limited to popular, open-source end-to-end frameworks and excludes record-and-playback solutions. The framework in the thesis is limited to applying best practice and discussing the potential of selectively integrating industrial standards to further enhance system reliability and performance in future iterations.

These industrial standards are used as guidelines and benchmarks for evaluating the framework; however, the implementation of these guidelines is beyond the scope of this thesis.

The remainder of the thesis is organized as follows. Chapter 2 provides a comprehensive literature review of automated HMI testing, covering software-testing fundamentals, end-to-end (E2E) practices, and relevant international standards. Chapter 3 presents the project overview, objectives and insights gained from stakeholder consultations. Chapter 4 describes the framework design, explaining evaluation criteria, tool comparison, overall architecture, CI/CD, alignment with industry standards and best practices. Chapter 5 documents the practical implementation, including server setup, test development and metrics gathered from real-world test runs. Chapter 6 discusses and analyses the framework’s performance, scalability and current limitations. Chapter 7 concludes the thesis by summarizing key findings, highlighting contributions, and reflecting on the framework’s impact. Finally, Chapter 8 proposes directions for future work—to further enhance the robustness and usability of automated HMI testing in safety-critical environments.

2 Literature review

2.1 Software testing

Software testing is a process in which to find issues of a software application, verifying that it meets all technical and non-technical requirements and ensuring fitness for its intended use (Anon., 2021). It includes verification, which checks that the product has been built correctly according to its specifications, and validation confirming that the finished product is the right one for its intended purpose. Therefore, software testing helps to reduce the risk of costly failures and increase the user satisfaction level (Rex, et al., 2009). Empirical study reveals that the later the defects are found in the development process, the more dramatical escalation of the cost of correcting them. Defects found in production may cost one hundred times more than found during the early stage of the development (Rex, et al., 2009).

There are multiple testing approaches that are categorized by executing method which are manual and automated - and by the testing scope, including functional and non-functional testing (Vahid & Mika, 2016). Manual tests are performed by human testers, remains essential for evaluating user experience, accessibility, and domain specific behaviours that defy automation. On the other hand, automated testing utilizes programmed test cases and specialised testing frameworks to execute repeatable tests rapidly, making it indispensable for regression validation and continuous integration pipelines (Vahid & Mika, 2016). Functional tests confirm conformance to requirements at the unit, integration, system, and end to end levels, whereas nonfunctional tests assess performance, scalability, security, and other quality attributes (Vahid & Mika, 2016). In this thesis, we focus on automated testing.

2.1.1 Automated testing

Testing of software has been remarkably evolving over the past few years, transforming from mainly manual approaches to highly automated frameworks supporting continuous integration and agile development.

One of the earliest and most widely adopted automated testing tools is Selenium WebDriver. Selenium gained attraction from community due to its open-source nature, support for various programming languages as well as cross browser integrating compatibility. The ability to simulate user interactions and running test scripts across environments made it a foundation of automated web application testing. However, despite its strengths, Selenium faces challenges when dealing with modern web interfaces that involve

dynamic content loading and asynchronous functions, leading to unstable and fragile tests that require frequent maintenance (Beata & Agnieszka, 2024).

In recent years, Microsoft's Playwright has emerged as a modern alternative as a response to some of Selenium limitations. Playwright has functions such as native auto-waiting, minimizing the risk of flaky tests, as well as multi-browser context simulation, assuring consistent behaviours across different browsers. Moreover, Playwright designs facilitate flawless integration with CI/CD pipelines, which enables more efficient and scalable test execution in agile development (Saad, et al., 2025).

Recent empirical studies have demonstrated that although the initial investment in automated test development can be substantially higher than manual testing, the potential for repeated execution and early defect detection results in significant cost savings over time, yielding a high ROI in the long term. Furthermore, prioritization models have been proposed to compare the manual testing effort against automation costs, thereby identifying which test cases should be automated to maximize efficiency. However, challenges such as technical debt and maintenance overhead remain significant impediments that must be addressed to ensure long-term reliability (Rudolf, et al., 2014)

Although automated testing has great advantages, automation requires significant initial investment and ongoing effort of maintenance. Automated test case can become brittle, being prone to false positives when application changes alter element identifiers and may impart a false sense of security if coverage is incomplete, or tests are not routinely updated. Therefore, the development team must weight automated and manual efforts according to risk, complexity, and resource constraints, embedding testing activities early and continuously throughout the development life cycle to have the highest yields return on investment (Rex, et al., 2009).

2.1.2 Criteria for Automated Testing

Deciding which application, we need to automate testing is critical for cost-effective. Research (Vahid & Mika, 2016) identify five factors that favour automation:

- The life cycle of software is long
- The upfront investment of automation is manageable
- Development process generates a relatively big number of builds
- Automated analysis of the test output is straightforward
- Tests need to run against multiple hardware and software environments

Selecting an appropriate automation tool and framework requires understanding their language support, browser compatibility, learning curve, and community backing. Therefore, expertise in testing tools is fundamental to effective automation.

2.1.3 End-to-end testing

Nowadays, web applications are more and more complex in architecture and modules, which are built on multiple layers and distributed across different devices (Maurizio, et al., n.d.). End-to-end (E2E) testing is designed to test the software application modules, which is one of the most popular test strategies in software development and performed along with unit and integration testing (Maurizio, et al., 2016)

E2E testing occupies the highest tier of the testing pyramid. It validates comprehensive user workflows in a production-like environment. Contrasted from unit and integration testing, which validate individual components or discrete interfaces. E2E test mimic the practical user interaction flow with the software throughout the whole technology stack. It includes user interface level to services, APIs, and data storage, to assure that the software behaves as expected and all the components and subsystems are correctly tightened and exercised together as a whole (Pittet, n.d.). Therefore, E2E testing is essential for finding the defects that emerge when modules integrate and interact, providing the strongest assurance of system reliability and user satisfaction.

A critical prerequisite for robust E2E automation is reliable element localisation. DOM based localization leverages the browser's Document Object Model to identify UI elements by attributes such as id. This flexibility enables straightforward test authoring but renders the tests fragile to source-code changes. (Maurizio, et al., 2016). Alternative strategies, including coordinate based and visual pattern localisation, trade maintainability for resilience against UI refactoring but often incur higher implementation complexity and lower performance.

2.1.4 Best practices for E2E testing

Strategic prioritization and meticulous execution are essential for an E2E testing program to be effective. First, the test scenarios have to be selected based on risk basis, focusing on business-critical user work flows, in which a failure could lead to catastrophic operational or financial impact (Margot & Mallory, 2020) (Anon., 2020). Test should be designed as small, self-contained units that are independent of one another, enabling rapid error diagnosis and minimizing cascading failures. Cooperation among stakeholders,

including product management, user experience designers, and software engineers, is crucial to assure that test cases accurately reflect end user behaviour and evolving requirements (Anon., 2020).

From technical point of view, stability and reproducibility are significant. In order to improve flakiness caused by dynamic content and asynchronous functions, automated test must use reliable locator strategies such as explicit data-testid attributes and explicit synchronization constructs (Vahid & Mika, 2016). The test should be executed in a production-like environment to eliminate environment variability and assure that test results are reproducible and meaningful. Finally, E2E testing framework should be integrated into CI/CD pipeline, enabling test cases to be automatically executed on every change of the codebase. That brings the benefits of fast regression detection and a rapid continuous feedback loop, which is greatly beneficial for agile delivery.

2.1.5 Test case design

E2E testing is involved around the five core stages, including test scenarios design, test case scripting, execution, valuation and result reporting, which together construct a continuous feedback loop that pushes ongoing improvement in software development (Vahid & Mika, 2016). This lifecycle ensures that defects introduced at any stage of development are detected as early as possible, reducing cost of correction and accelerating release speed.

A test scenario is a structural specification of predefined conditions, inputs, test cases and expected result designed to verify a slice of application functionality (Devaraj, 2025). Well-designed test scenarios define testing goals before any implementation, reducing ambiguity and improving maintainability as well as defect finding ability. Other than striving for exhaustive coverage, developers should decide which scenarios are needed for test coverage, based on business impact and user criticality. That would ensure that the most critical workflows receive the highest level of assurance.

In general, test cases should be atomic, meaning they should have one simple objective that tests one function at a time. Moreover, each test should be repeatable and self-contained, one test does not have dependency on the status of the other ones, minimizing interdependencies and simplify root cause analysis when failures occur.

One of the five core stages of test is test scripting, translating test case definitions into executable code that drives the application under test via a chosen framework. Semantic DOM selectors, which is considered as reliable element localizing solution, and clear synchronization constructs such as auto

wait are essential to minimize flakiness caused by dynamic content and asynchronous operations (Maurizio, et al., n.d.) (Anon., 2020).

Execution of E2E test suites is performed within the chosen automation framework, with test results are captured and logged for post analysis. Tests are often carried out in a test-dedicated environment, ideally a controlled replica of production, which enables consistent, reproducible outcomes and isolation from development or live data. Standard practice separates development, testing, and production environments to prevent unintended side effects. Testing environments should have ability to reset of test data between test runs to maintain reliability and repeatability. Evaluation of test result separate true failures from false positives. Finding and improving flaky tests is significant to maintain a certain level of confidence in the test process, along with CICD dashboards insights analytic on failure trends and stability metrics over a course of time (Jetbrains, n.d.). Finally, a comprehensive report, delivering test result, execution duration, and failure artifacts (screenshots/videos) in a format user friendly format, allowing stakeholders to classify issues and continuously monitor software applications health (Schmitt, 2024)

2.1.6 Human–Machine Interfaces in Safety-Critical Systems

Human–Machine Interfaces (HMIs) is playing a significant role in operation of safety-critical industrial systems, serving as the most important link through which operators interact with complex control systems. In industrial systems, HMIs are responsible for transforming raw sensor data into actionable information, thereby enabling real-time decision-making that is crucial for maintaining operational safety (Chuanzan, et al., 2020).

The design of HMIs in power plant is remarkably challenging due to the need to balance technical functionality with ergonomic and usability requirements. Research has shown that poor interface design can significantly contribute to operator failure, with evident historical incidents such as a series of mistakes leading to the Chernobyl disaster, revealing the consequences of inadequate system feedback and poor design (Chuanzan, et al., 2020). Therefore, a robust HMI design must emphasize clarity, consistency, and intuitive operation to reduce the risks and improve system performance.

International standards lay a solid foundation for resolving these challenges. One of those standards, ISO 9241-210 frameworks human-centered design principles encouraging user-friendly and ergonomically sound interfaces, which assures that HMIs designs meet the cognitive and physical capabilities of their operators (ISO, 2019). ANSI/ISA-101.01-2015 lays out the critical requirements in designing HMIs, especially emphasizing safety, clarity, and

effective operator interaction (Automation, 2015). Although these standards offer comprehensive guidelines, their full implementation in any project can be challenging. However, they serve as a benchmark for evaluating and improving HMI designs in safety-critical environments (Eduard, et al., 2017). Recent research continues to study innovative approaches to improve HMI performance, focusing more on integrating advanced technologies and automation to reduce human error. These studies, emphasize the importance of systematic test and continuous evaluation HMI to assure that they not merely meet all the technical requirements but also effectively support operator performance in real-work conditions (Beata & Agnieszka, 2024) (Chuanzan, et al., 2020).

2.2 Continuous Integration and Continuous Deployment in Automated Testing

Continuous Integration (CI) and Continuous Deployment (CD) have become foundational practices in software development, particularly in agile and DevOps environments in the recent years. The integration of automated testing frameworks with CI/CD pipelines allows the immediate performance of test cases on every code commit event, reducing the time between code changes and the detection of issues (Jetbrains, n.d.). The continuous loop of feedback is crucial for maintaining the stability of complex systems such as web-based HMIs, where any delay in detecting issues can have significant operational downtime.

CI/CD pipelines facilitate not merely the automation of test performing but also the standardized test report management, providing insight into key performance measures such as the ratio of pass/fail tests, failure occurrences and execution duration. This reporting architecture is significant in monitoring software quality over a course of time and supports the iterative improvement of both the testing process and the overall system. Moreover, containerized testing environments, enabled by technologies, for instance, Docker, help ensure consistency across different stages of deployment, further enhancing the reliability of the testing process (Jetbrains, n.d.).

Modern testing tools, such as Microsoft's Playwright, are designed with CI/CD integration as one of a must-have prerequisite. Playwright's native support for automated workflows allows for a smooth incorporation into CI/CD systems such as Bamboo, GitHub Actions, or Bitbucket pipelines. It is crucial for reaching the stage of fast regression detection and effective maintenance in rapid pace of modern software development. Effective CI/CD integration not merely accelerates up feedback loops but also being significant contributing factor in improving test coverage and reliability of the whole system (Beata & Agnieszka, 2024) (Jetbrains, n.d.)

2.3 Data Generation and Reporting in Automated Testing

Automated testing frameworks not only perform tests repeatedly but also enable generating report of performance data, which is important component for assessing system reliability and the effectiveness of the test. Recording the most important technical metrics like uptime, error rates, is important to maintain continuous, quantitative feedback. These technical measures provide developers insight into health of the system over a course of time and identify trends that may indicate emerging issues before they escalate into serious problems (Chuanzan, et al., 2020) (Dorothy, et al., n.d.)

In practice, automated test reports should be structured to provide a clear, and actionable information of the software's behaviours under testing. Best practices in test reporting suggest that metrics should be presented in standardized formats in order to facilitate integration with CI/CD pipelines and to be readable by both technical Comparative Analysis of Manual vs. Automated Testing Effort and Effectiveness and non-technical stakeholders. These reports generally include summary statistics for each test execution, along with detailed logs that document individual test results, execution times, and any encountered errors (Jetbrains, n.d.). This level of reporting ensures that each test run can be analysed independently, enabling teams to quickly isolate and address any issues without the need for aggregating data from multiple test cycles. Effective reporting design are essential for continuous improvement. Reports should be designed to trigger alerts when important thresholds are crossed, for instance, if error rates exceed acceptable limits or if response times degrade over consecutive test cycles. By integrating these best practices into test report design, teams can ensure that the automated testing framework not only detects defects promptly but also provides insights that guide iterative enhancements in both the software and the test process (Eduard, et al., 2017)

2.4 Comparative Analysis of Manual vs. Automated Testing Effort and Effectiveness

Research consistently reveals that automated test, while requiring a higher initial investment in terms of time and resources, can gain significant long-term benefits compared to manual testing. Early research in software testing highlights that manual testing, despite its strengths in exploring ad hoc scenarios and assessing user experience, is labour-intensive and prone to human mistake (Rex, et al., 2009). As software systems have grown in complexity, the limitations of manual testing, such as inability to scale and its inconsistent repeatability, have become increasingly apparent.

Several case studies, particularly in industrial control environments, have compared the cost, effort, and defect detection capabilities of manual versus automated testing. These studies indicate that automated tests, once developed, can be executed repeatedly with minimal additional cost, thereby significantly reducing the overall testing effort over time. For example, studies of industrial control software have found that automated test suites can deliver the same level of code coverage as manual testing, yet they operate in a fraction of the time, leading to earlier defect detection and lower long-term costs (Eduard, et al., 2017). Such findings underscore the importance of prioritizing test cases for automation, ensuring that tests with the highest potential for repeated execution and defect detection are selected first.

In addition, models for prioritization have been proposed that evaluate test cases based on the manual effort required versus the anticipated benefits of automation. These models help organizations identify which tests should be automated to maximize ROI. By quantifying factors such as the cost of manual execution, the frequency of test runs, and the potential impact of undetected defects, these models provide a structured approach to test selection and scheduling. Although the cost of developing automated tests is generally higher, the cumulative savings in execution time and improved defect detection make automation a strategic investment, particularly in safety-critical environments where the cost of failure is exceptionally high.

3 Case Study

This project delivers an automated testing framework for a web-based HMI interface, which is a critical control and monitoring tool used across power plants. The interface uses the Atvise Builder, which exports production-ready JavaScript files that run in Chromium. Because those exports lack a source-code structure and are not suitable for unit and integration testing, teams have relied on manual validation. This testing approach is not only labour-intensive and error-prone but also repeatedly consumes valuable resources every time a change is made.

The HMI must support a variety of plant configurations by adapting its layout and behavior in two ways: by re-exporting updated files from Atvise or by invoking internal browser APIs at runtime. Effective testing therefore requires not only verifying that the graphical interface renders correctly but also confirming that each configuration change produces the intended functionality. Manual testing for various versions of HMI is unsustainable.

To address this, we embed our E2E suite into Bitbucket via a Bamboo plan that spins up Docker containers on each commit. Those containers run Playwright against the deployed HMI and block any merge if tests fail. This automation reduces human error prone with faster feedback as well as frees engineering resources for more important engineer tasks.

3.1 Motivation

The motivation of this project is from the limitations of our current testing methods. Manual testing of Atvise-exported JavaScript reveals two major challenges. The first challenge is that the building artifacts are monolith architecture and lack modularity codebase, making unit tests impractical. Other issue is that the HMI's dynamic configuration options multiply the number of manual test scenarios required. In a safety-critical environment, these gaps endanger product integrity and amplify the risk of undetected defects. Automated E2E testing framework can ensure that every change, from simple UI modification to bigger update, is reliably verified before deployment.

3.2 System Context

The HMI operates within a complex, configurable ecosystem. Atvise exports form the core interface, but runtime behaviours can also be altered via proprietary browser APIs. This dual-configuration mechanism supports scenarios ranging from small turbine controls to full-scale grid management.

Consequently, our framework must verify that the user interface renders and responds correctly under every configuration, confirm that configuration API calls drive the intended system behaviours, and integrate smoothly with existing build infrastructure (Bitbucket, Bamboo, Docker). Automated, end-to-end testing is therefore essential to guarantee production quality across all supported plant configurations.

3.3 Objectives and limitations

The primary goal of the project is to validate both the visualization of the interface and the configuration logic of the HMI on each commit. The solution must validate that screens, transitions, and user interactions work as designed, that critical internal API calls make the expected HMI transition. These tests play an important role in determining whether a commit should be merged based on whether the critical features pass predefined quality criteria.

However, we also faced several limitations. The JavaScript exported by Atvise is complex, bundled code that is not structured for unit testing, forcing reliance on end-to-end tests even more. The HMI server's licensing mechanism adds an operational overhead that must be automated so that we can run multiple containers for parallel tests. The complexity of configuration logic requires a set of flexible test cases which are capable of handling various workflows and edge cases. Moreover, the requirement is also that the solution must run within our existing Bamboo infrastructure and additional platforms or tools should be avoided if possible.

To handle those challenges, the project involves extensive discussions and interviews with the HMI engineers as well as system engineers. These consultations help refine the testing requirements by identifying which user scenarios and critical API functions are “must-have”. With their help, we also define qualitative criteria such as execution duration, test coverage as benchmarks for the framework. The understanding of both the operational environment and the technical limitations helps laying the solid foundation the framework architecture design.

3.4 Stakeholder consultations

The design of our testing framework was grounded in a series of in-depth consultations with key stakeholders, starting primarily with HMI developers and system engineers, whose insights ensured that our solution addressed practical challenges. In the first workshop, we discussed the existing manual testing process, mapping each step from code export through deployment

and identifying the most time-consuming and error-prone workflows. Moreover, system engineers demonstrated their daily work related to HMI configuring and how complex configuration combinations and ad-hoc API calls currently demanded extensive human verification. These discussions emphasized the critical need of an automated test framework.

Extending from the understandings of the first meeting, the second session focused on defining the precise testing scenarios that must be covered. We walked through user journeys, such as how users navigate between screens, trigger alarm dialogs, and apply configuration profiles via internal APIs, to capture both UI and API interactions. This collaborative modelling of test scenarios validated that the Page Object Model would accommodate the HMI's dual configuration paths and highlighted the necessity of end-to-end scripts that could both manipulate runtime settings and verify visual changes. This discussion laid the foundation for the E2E framework requirement, which is used as the benchmark to select the correct tools.

In the final discussion, we presented our proposed CI/CD integrated architecture and solicited feedback on integration points with Bitbucket, Bamboo, and our Ubuntu server environment. Stakeholders with the proposal that embedding test execution directly into the pull-request workflow (PR) would deliver the fastest feedback loop as well as align with existing operational practices. Engineers also emphasized the importance of clear reporting, linking build artifacts back to specific commits and configuration versions. It allows developers and operators trace any failure back to its root cause.

These three iterative workshops not only refined the technical requirements, defining which UI elements and API calls are non-negotiable test targets, but also shaped development team experience goals for test reporting and feedback. By engaging stakeholders at every stage, we have ensured that the framework is both technically appropriate and a solution for the practical challenges we have.

With requirements and constraints firmly established through stakeholder workshops, Chapter 4 describes our architecture and the technical realization of this testing framework.

4 Framework Design

In safety-critical power-plant environments, every HMI change must be validated in a production like setting before released. This chapter describes the design of our automated E2E testing framework, which integrates Bitbucket, Bamboo, Docker, and Playwright, to guarantee fast, reliable feedback on every commit and prevent regressions in Human-Machine Interfaces.

4.1 Overall Framework Architecture

The proposed solution automatically validates web-based Human–Machine Interfaces (HMIs) by embedding e2e tests into a Bamboo integrated CI/CD pipeline. Each change commit in HMI repository in Bitbucket triggers a test plan, ensuring consistency and reproducibility by encapsulating all dependencies and environment configurations.

The automated testing layer is built around the Playwright framework, which we drive through a TypeScript interface to design, execute and maintain our E2E test cases. By leveraging Playwright’s built-in cross-browser support, we validate our HMI with our customized browser within our existing Selenium-based executable. This layer is also where test reporting is managed, deciding what data to capture, its format and stored place.

Supplement for the automated testing layer is the continuous integration, where we use Bamboo to orchestrate our test plan. Whenever a developer pushes or merges code in Bitbucket, Bamboo picks up the new commit, checks out the exact hash and spins up a fresh test environment. Playwright then runs the full suite, producing an interactive HTML report with step-by-step details alongside JUnit and JSON artifacts, plus concise console logs. Bamboo archives all these artifacts and automatically links them to the corresponding pull request, effectively blocking any merge if a test fails.

Once test cases run complete, Bamboo immediately pushes real-time notifications to the development team, complete with direct links to the detailed HTML reports. This fast feedback loop keeps everyone aware of regressions and speeds up our troubleshooting cycle.

We ensure scalability by using containerization technology, in which tests execute concurrently on multiple Docker agents, and our CI/CD pipeline automatically adjusts resource allocation to match the workload. By defining our environments in Docker, we guarantee that our testing environment reflects environment in production closely as possible.

Design decisions support our research goals by leveraging Playwright’s robust handling of dynamic content and continuous integration, aligning with industry best practices for a reliable HMI testing. This integrated architecture delivers continuous, production-grade feedback and directly addresses the optimal architecture questions at the heart of this thesis.

4.2 Tool Selection and Comparative Analysis

During our stakeholder engagement meeting, we establish a set of evaluation criteria that serve as the basis for comparing test automation frameworks. We categorize these criteria into two groups: primary requirements, which outline the essential, mission-critical functions our solution must provide, and secondary requirements, which cover supplementary features that improve usability and facilitate future extension.

4.2.1 Tool selection criteria

We derived our primary requirements (refer to table 1) from hands-on observations and stakeholder consultations, ensuring the framework can execute custom JavaScript in the browser context while remaining compatible with Chromium-based browsers. It must accurately simulate click actions, implement wait strategies for UI elements and their dynamic properties, verify both static and localized text, and integrate smoothly with CI/CD pipelines to enforce automated gating of code changes. (Rex, et al., 2009) (Chuanzan, et al., 2020).

Table 1. Critical requirements for E2E framework.

Parameter	Critical Requirements for Our Project
Custom JavaScript Execution	Must support the execution of custom JavaScript within the browser context to simulate user actions and trigger internal API calls.
Chromium-Based Browser Compatibility	Must ensure compatibility with our customized browser within our existing Selenium-based executable
Click Simulation	Must include reliable click agents that accurately simulate user interactions, such as clicking buttons and links.
Waiting Mechanisms	Must incorporate mechanisms to wait for UI components and their values to load and update, ensuring reliable execution in dynamic web environments.
Content Verification and Translation Support	Must offer capabilities to verify UI content, including support for translation features to confirm correct localization.
CI/CD Integration	Must integrate seamlessly with CI/CD tools, such that test results are automatically linked with Bitbucket commits to gate code merges.

The secondary requirements (table 2), formed from the broader literature in test automation (Garousi & Mäntylä, 2016; Antonczak & Bylina, 2024), include factors such as ease of learning, extendibility, reporting capabilities, and cost-effectiveness. While not strictly required, these factors add some value by reducing maintenance overhead and supporting the long-term scalability of the testing framework.

Table 2. Secondary requirements for E2E framework.

Parameter	Critical Requirements for Our Project
License	Open source – free to use
Record, trace on failure	Record frame, video, traces of error
Supported programming language	Javascript and Python
Report	HTML and CICD friendly report

To apply these criteria, we evaluated five leading end-to-end test frameworks. Cypress is good at fast, JavaScript-centric development with built-in

retries and an interactive debugger. Selenium can bring a mature, multi-language ecosystem and broad browser driver support, though it does require more explicit waits. Besides that, Puppeteer provides deep, scriptable control over headless Chrome/Chromium, making it ideal for Chrome-only workflows. Nightwatch builds on Selenium WebDriver to simplify syntax and CI integration. Playwright, by contrast, unifies cross-browser coverage, robust auto-waiting, built-in tracing, and concise TypeScript APIs into a single package. For detailed profiles of each tool's advantages and limitations, please see Appendix A. The next section presents the weighted scores for each framework across our criteria.

4.2.2 Comparison

The comparative analysis table 3 quantifies how the most popular four automated testing frameworks, perform against our key evaluation criteria. Each row of the table represents a specific requirement (primary and secondary requirements), with a designated weight reflecting its importance. Each framework is scored on a scale of 1 (poor) to 5 (excellent) for each requirement, and the individual scores are multiplied by the respective weights. The sum of these weighted scores produces an overall total for each tool.

The comparative analysis table helps to highlight the strengths and weaknesses of each testing frameworks relative to our project's key evaluation criteria. In our evaluation, we focused on requirements such as the ability to execute custom JavaScript, robust support for Chromium-based browsers, reliable click simulation, effective waiting mechanisms for dynamic content, comprehensive content verification (including translation support), continuous CI/CD integration. Moreover, supported programming languages, JavaScript and Python is also a significant evaluating point aligned with our team expertise. In addition, factors like ease of learning, extendibility, reporting capabilities, and overall cost-effectiveness were also considered.

Table 3. Framework evaluating table.

Evaluation Criteria	Weight	Cypress	Cypress Weighted	Selenium	Selenium Weighted	Playwright	Playwright Weighted	Nightwatch	Nightwatch Weighted
Custom JavaScript Execution	5	5	25	4	20	5	25	4	20
Chromium-Based Browser Compatibility	5	5	25	5	25	5	25	5	25
Click Simulation	4	5	20	4	16	5	20	4	16
Waiting Mechanisms	5	5	25	3	15	5	25	3	15
Content Verification & Translation Support	4	4	16	3	12	5	20	3	12
CI/CD Integration	5	5	25	4	20	5	25	4	20
Supported Programming Languages (JS & Python)	3	3	9	5	15	4	12	3	9
Subtotal Primary	-		145 (2)		123 (3)		152 (1)		117 (4)
Ease of Learning	3	5	15	3	9	5	15	3	9
Extendibility	3	4	12	5	15	5	15	4	12
Reporting Capabilities	3	5	15	4	12	5	15	4	12
Cost-Effectiveness	2	5	10	5	10	5	10	5	10
Subtotal Secondary	-		52 (2)		46 (3)		55 (1)		43 (4)
Total Score	-		197 (2)		169 (3)		207 (1)		160 (4)

When comparing the frameworks, it's evident that while all of them meet the baseline requirements, differences are in their advanced features and overall usability. Selenium is recognized for its mature ecosystem and excellent multi-language support, which makes it appealing for the team to need a versatile solution. However, its approach to handling dynamic content often requires additional effort with

explicit waiting conditions and more complex locator strategies. This can make maintenance more cumbersome in our highly dynamic HMI testing environments.

Cypress, with its intuitive API and strong native support for JavaScript, is great in simulating user interactions and offers automatic waiting mechanisms that simplify the test development process. Its reporting and integration with CI/CD pipelines are robust, making it attractive for iterative testing environments. However, its native focus on JavaScript, without strong Python support, is a limitation.

Nightwatch, which builds upon Selenium, provides reliable automation and user interaction simulation but unfortunately inherits some of Selenium's limitations when it comes to handling dynamic UI changes. Its design is straightforward if a team already invested in the Selenium ecosystem, yet it does not introduce the modern improvements seen in more recent frameworks.

Playwright distinguishes itself by combining the best attributes from the other frameworks. It natively supports custom JavaScript execution in a streamlined way and provides a sophisticated, built-in auto-waiting mechanism that minimizes test flakiness, which is a crucial component in dynamic HMI testing. Furthermore, Playwright's strong support for Python enhances its flexibility while maintaining robust native capabilities in JavaScript. Its friendly integration with CI/CD systems through multiple reporting formats is a good contributing factor to a fast continuous feedback loop.

4.3 Framework architecture

Building on the project goal laid out in section 3.3, the framework testing layers are integrated with our CI/CD pipeline. The source code of HMI is managed in Bitbucket, where each code commit immediately triggers a Bamboo test plan. This plan orchestrates the entire testing process, starting from establishing a secure SSH connection to a dedicated Ubuntu test server.

By encapsulating all necessary dependencies, including Node.js, Playwright, browser engines, and custom libraries, within a pre-built Docker image, the framework minimizes the setup time and eliminates residual state between test executions, ensuring consistency over test runs. Docker launches the container (see 4.4.3), in which Playwright runs our end-to-end suite against the deployed HMI application.

These tests are designed to simulate practical user workflows and critical component interactions of the HMI. They cover scenarios such as screen transitions, error handling, user interaction patterns, and complex HMI configuration process via internal API response validations. The test cases are developed in a modular fashion, following best practices Page Object Model for better maintenance as well as scalability.

As the tests run, detailed logs, screenshots, and trace captures are generated. The results are compiled into structured reports in multiple formats (see 4.4.4) and are stored as build artifacts within Bamboo. Each test run is assigned a unique URL, providing immediate access to the comprehensive test results.

Finally, the framework integrates tightly with Bitbucket. Based on the test outcomes, Bamboo communicates back to Bitbucket by posting a message on pull requests with a direct link to the test manifest. If tests fail, the corresponding pull request would be blocked from merging until the issues are resolved. This immediate feedback loop ensures that only code meeting our quality standards is integrated into production, thereby minimizing potential risks of bugs in production.

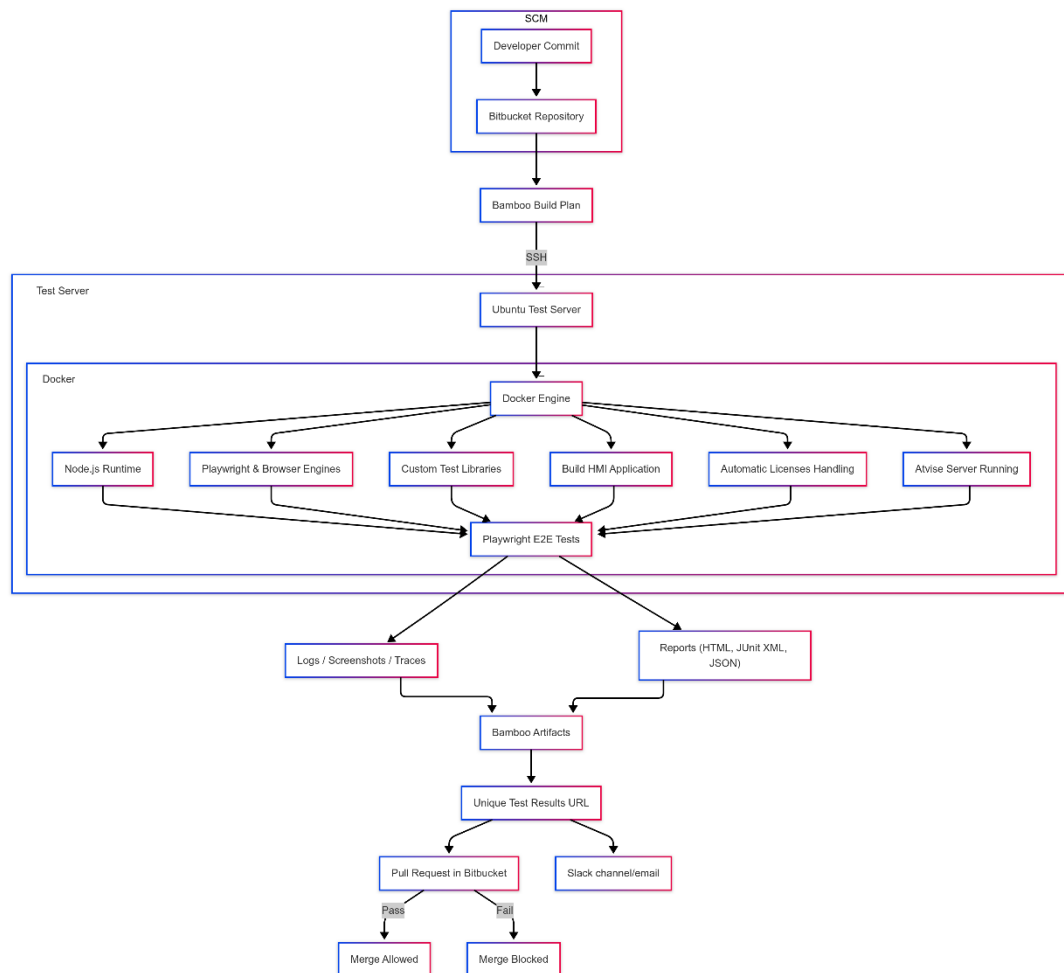


Figure 1. CI/CD-integrated E2E testing pipeline for the HMI application

4.3.1 Playwright

Our Playwright configuration is designed to support test execution environment within our automated testing framework with the scalability in mind. This section outlines all key technical details and choices that ensure our E2E tests are executed reliably in a containerized environment and well integrated with our CI/CD pipeline.

First, Playwright is configured to run in headless mode, which is essential for resource conservation and optimal performance within the pre-built Docker image. The configuration below lives in `playwright.config.js` (refer to Appendix A)

Running in headless mode significantly speeds up test execution while still providing consistent behavior across different runs. The configuration defines a fixed viewport of 1280 x 720 pixels, which matches the exact resolution of our HMI display, to ensure that the tests accurately simulate the user interface used in production. Furthermore, the configuration specifies the use of the Chromium browser, particularly via the Chrome channel. This choice supports our requirement for stability and compatibility, given that our HMI application is heavily optimized for Chromium-based environments. We ignore HTTPS errors so that self-signed certificates do not halt critical tests.

Detailed logging is an important aspect of our test strategy. Our setup uses Playwright's `test.step()` to highlight key actions in each test case. This creates clear narrative in the HTML report, making it easier to troubleshoot when failures occur. The configuration also enables the trace: 'retain-on-failure' option. In the event of a test failure, this feature captures comprehensive trace data, including browser state snapshots, DOM structure at failure points, network log details, and console outputs. Although the setup currently captures discrete data points, it lays the groundwork for potential future enhancements such as selective video capture and historical data. Our configuration specifies three primary reporters:

- An interactive HTML reporter that creates detailed, navigable reports with integrated screenshots and trace logs.
- A JUnit reporter that outputs XML-formatted data for friendly integration with our CI system, Bamboo, which relies on these reports for automated build status updates.
- A JSON reporter that produces structured data for further analysis and trend monitoring.

Besides, flexibility is built into our configuration through the use of environment variables. Inside Docker container, the

PLAYWRIGHT_BROWSERS_PATH environment variable is set, allowing us to specify a custom browser executable path. This is especially useful for accommodating updates or using particular browser builds without altering the core configuration file. Moreover, the configuration includes dynamic settings via Node.js, including reading a list of test cases from an environment variable (TEST_CASE) so that specific test suites can be targeted during different runs.

The main configuration is defined in the playwright.config.js file, including all the setup details discussed above. It also specifies the test directory, maximum number of worker processes, and retry logic to ensure parallel test execution capability and balance resource use at the same time. The configuration is supported by a corresponding TypeScript configuration (tsconfig.json), which enforces strict type-checking and ensures that our test code follow to modern JavaScript standards (ES2021).

4.3.2 Continuous Integration and Continuous Deployment

Our CI/CD integration process is the backbone of our automated testing framework orchestration, ensuring that every change to the HMI application is immediately validated before merging. When a developer commits code to Bitbucket, a webhook immediately triggers a preconfigured Bamboo build plan. This build plan orchestrates the entire workflow, from establishing secure connections to remotely provisioning environments, all the way through test execution, artifact collection, and feedback integration.

The process begins when a code commits triggers Bamboo. In our Bamboo YAML configuration, the plan first checks out the repository and sets up necessary build variables.

The next steps in the sequence, Bamboo establishes a secure SSH connection to a dedicated Ubuntu test server. This is accomplished by decoding an AWS SSH key and adding the host to the known_hosts file to ensure secure connectivity. Below is an excerpt that shows this SSH preparation (refer to Appendix B)

Once the SSH tunnel is in place, the environment is set up using Docker. A pre-built Docker image, configured with all necessary dependencies (Node.js, Playwright, browser engines, and additional utilities), is launched on the remote server.

This containerized approach guarantees a consistent environment for each test run. It also reduces setup overhead. Once the container is up, it

automatically pulls the latest version of the HMI application and begins executing our test case of E2E tests with Playwright.

As the tests execute inside the Docker container, detailed logs, screenshots, and trace data are generated. These artifacts are stored in the designated playwright-report directory, which is mounted as part of the build environment so that Bamboo can collect them automatically. Once the test run is completed, the Docker container is dropped immediately, ensuring no residual state remains.

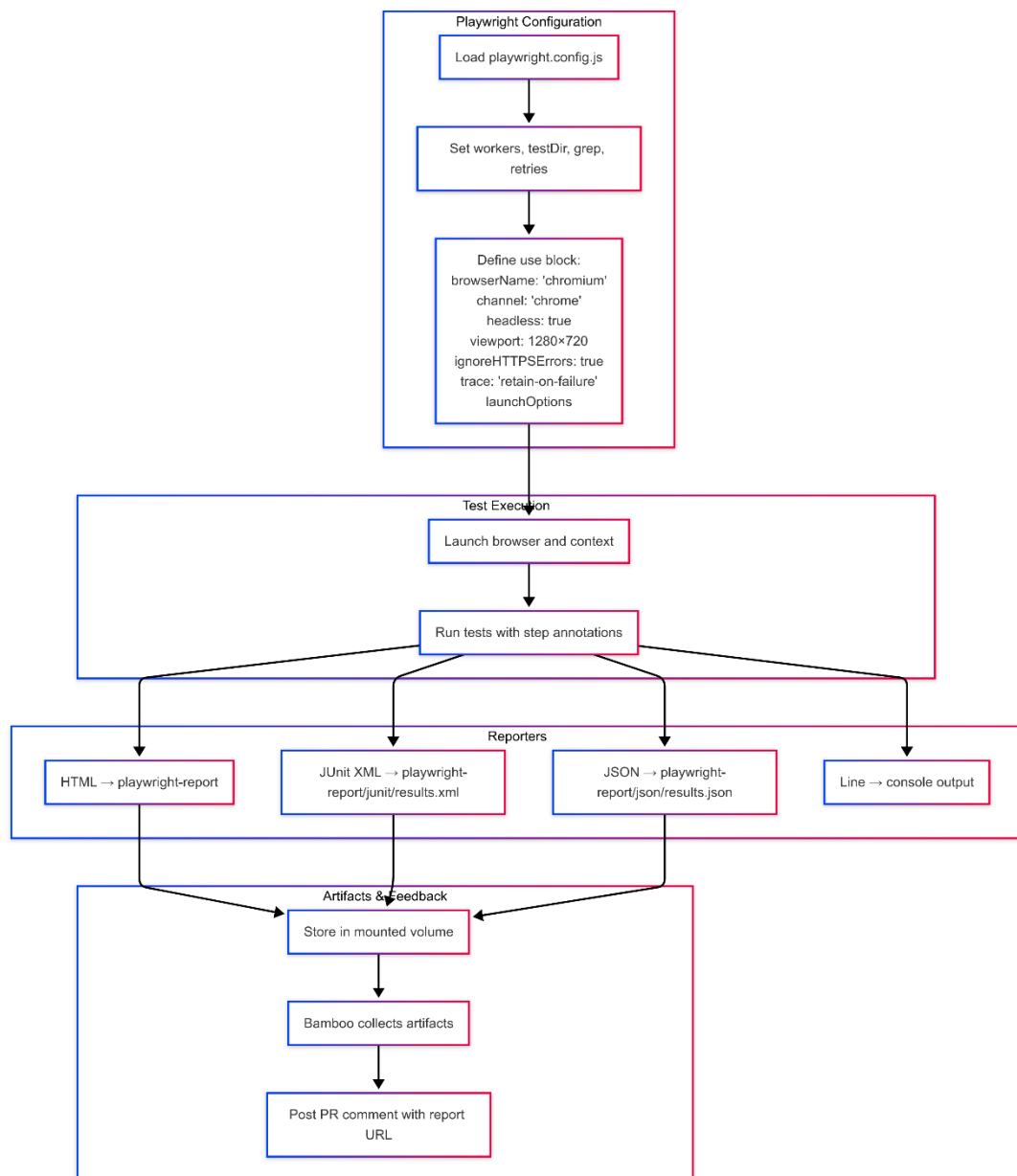


Figure 2. Playwright CI/CD Test Execution and Reporting Pipeline

Upon completion of the tests, Bamboo collects the test artifacts (HTML, JSON, and JUnit reports) into a build manifest. A unique URL is generated for each test run, making the detailed test report accessible immediately. This URL is then integrated back into Bitbucket's pull request workflow. Because the build was originally triggered by a Bitbucket push or pull-request event, Bamboo automatically exposes the corresponding commit SHA and pull-request ID in environment variables (`bamboo.repository.revision` and `bamboo.bitbucket.pr.id`). These variables serve as the definitive link between the test results and the exact code change under review (refer to Appendix C).

Besides the Bitbucket update, the same script sends a JSON payload to the team's Slack incoming-webhook URL (<https://hooks.slack.com/services/<...>>). This payload includes the plan key, build number, commit SHA, pull-request link, and manifest URL. In Slack, the notification appears in the designated channel as

```
[<PLAN_KEY>-<BUILD_NUMBER>] E2E tests for PR #<PR_ID> (commit <COMMIT_SHA>) have <status>. View full results → <manifest URL>
```

This tight feedback loop ensures that if any test fails, the PR merge is automatically blocked, and developers and stakeholders receive immediate notification along with a direct link to the test results. In contrast, a passing test run allows PR to proceed. This integration not only enhances development agility but also maintains a high standard of quality control, essential for a safety-critical system such as a power plant HMI.

4.3.3 Docker image

Our test container is built upon Microsoft's official Playwright image (mcr.microsoft.com/playwright:next), which already includes Playwright, Node.js. We layer on essential utilities, including Git, tmux, build-essential, and Bash, and immediately cleaning up package metadata with `rm -rf /var/lib/apt/lists/*` to minimize image size and maximize caching efficiency. The Dockerfile sets the working directory to `/project`, ensures the Playwright binaries are executable, and switches to the root user to allow system installations and repository cloning (refer to Appendix C).

The core of our container orchestration is the `entrypoint.sh` script, beginning by echoing its start time and environment variables for easier troubleshooting. It then removes any existing checkout of the HMI code, clones the repository over SSH, and checks out the designated feature branch (refer to Appendix D).

Within the test/test-scripts directory, dependencies for test framework are installed via Yarn, customized Chromium executable is fetched. It use the PLAYWRIGHT_BROWSERS_PATH environment variable value as the path to find the browser executables. Besides, permissions must be grant to ensure Playwright has access to the browser binaries.

Once the code and dependencies are in place, the script returns to the project root to run the Webpack build, which is then used to spin up the HMI. The database file (nodes.db) and external assets (web and dist) are the core of HMI, which is output of Webpack build process, copied into a Atvise dedicated server directory. Finally, Playwright's end-to-end tests are executed in headless mode with tracing enabled, and any failures result in a non-zero exit code that Bamboo will detect and report.

In our Docker Compose setup, we mount the host's SSH keys into the container to permit Git authentication and bind the playwright-report directory back to the Bamboo workspace so that HTML reports, JUnit XML, and JSON results are automatically collected as build artifacts. Port 80 is exposed on the Docker host to allow the Atvise server running inside the container to handle API validations against a live HMI backend instance running on the host, Ubuntu server. All critical values like repository URL, branch name, browser binaries path, and report directories, are all injected via environment variables, providing full flexibility to switch branches, toggle additional diagnostics, or point to alternative browser builds without modifying the Docker image itself.

4.3.4 Report

Our reporting setup provides detailed visibility into each test run, offering both summaries and diagnostic information for thorough review. The main output is HTML report, which offers visual detailed overview of the test execution. This report summarizes the total number of tests executed along with the counts of passed, failed, and skipped tests, while also providing a step-by-step breakdown of each test scenario. Moreover, the report also include execution timings and step statuses, with embedded screenshots and trace captures for any failures when the 'retain-on-failure' setting is activated. Users can interactively navigate through this report, checking individual test cases to review the sequence of operations, which is particularly useful for finding issues and understanding system behaviours under various test cases.

In addition to the HTML report, our system generates a standardized JUnit XML report. This report is structured according to the JUnit schema, ensuring that it well integrates with our CI/CD. The JUnit output has key performance metrics at the test suite level, including the execution time for each test case, overall test counts, and detailed descriptions of any failures, stack traces and error messages. It enables automated build status reporting and facilitating rapid issue identification within CICD debugger. Moreover, this structured data serves as the backbone for aggregating test results across multiple runs, laying foundation for future improvement in which we collect and analyse metrics across multiple tests.

The last one is JSON report, offering detailed collection of metadata that captures the states of individual test cases. The JSON is highly compatible format, as it can be parsed by various external tools to extract custom metrics, visualize trends via dashboards, or even feed into advanced analytical systems. The raw, detailed data available in this format provides a foundation for performing retrospective analyses and continuous performance optimizations, ensuring that we can correlate test outcomes with changes in the application over time and over test cases.

During test execution, our framework simultaneously leverages a line reporter that outputs real-time, concise logs directly to the Bamboo CI/CD console. This live stream of updates is optimized to provide immediate feedback on the progression of the test suite, indicating when individual tests start, finish, or fail without overwhelming the user with excessive detail. The minimal overhead provided by the line reporter ensures that the critical information is available immediately, allowing developers and testers to quickly action overall system health as tests run.

4.4 Application of International Standards

The quality and usability of Human–Machine Interfaces (HMIs) in safety-critical environments are regulated by international standards designed to ensure that interfaces are both efficient and user-friendly.

In our framework design, although full compliance with these standards is not currently implemented, we have intentionally used them as benchmarks to guide our evaluation and set a foundation for future enhancement.

The first standard we researched in this study, ISO 9241 210, emphasizing a human-centred design approach that prioritizes usability and the minimization of user errors. It offers essential guidance on crafting interfaces that align with operator cognitive and physical abilities (ISO, 2019). Similarly, ANSI/ISA 101.01 2015 outlines detailed requirements for HMIs used in

process automation systems, focusing on clear communication, consistency, and safety, which are essential for preventing costly errors in high-risk operational settings (Automation, 2015).

In this section, we mainly assess how our current framework aligns with these international guidelines and identify specific areas where further integration could enhance both test design and overall system evaluation. By mapping our framework’s features against the key elements of these standards, we reveal that while our approach already meets many operational requirements, there are always opportunities to incorporate additional ergonomic and usability guidelines.

Although our primary objective in this project is to deliver a practical and efficient automated testing solution, establishing these standards as benchmarks validates our design choices and provides a clear roadmap for future enhancements. Importantly, this research lays the groundwork for a more complete integration of internationally recognized best practices into our testing framework, thereby ensuring that it can evolve to meet the highest safety and usability criteria in future iterations.

4.4.1 Standard ISO

ISO 9241 210 is standard outlining specifications and guidance for applying human-centred design at every stage of interactive system development (ISO, 2019). Contrast from many other standards broadly discussing usability, ISO 9241 210 is focused on ensuring that systems are designed from a human centred perspective. It requires that design must be grounded in a deep understanding of the users, their interactions, as well as the environments in which the system will be used. This requirement is crucial in safety critical settings where even small usability shortcomings can have dramatic consequences.

The standard begins by clearly defining the scope and terminology necessary for human centred design. It specifies key terms “usability,” “user,” and “context of use” in precise language so that all stakeholders have a common understanding (ISO, 2019). The standard sets the stage for an iterative design process where every decision is informed by knowledge about the users and the situational context. This foundational premise is particularly valuable in our domain, where HMIs must reliably convey critical information to operators under varying conditions.

The central of ISO 9241 210 are the human centred design principles. The standard states that the design is based on an explicit understanding of users, tasks, and environments. This is achieved by gathering user requirements,

analysing the context of use, and ensuring that the design process is iterative. The standard further highlights that users must be actively involved throughout the design and development process. This involvement is not limited to simple usability tests but extends to continuous user centred evaluation, whereby iterative feedback from representative users refines the design until the solution meets both functional and ergonomic requirements. Such iterative evaluation and refinement are emphasized as fundamental to minimizing user error and improving overall system performance (ISO, 2019).

Besides that, another aspect of ISO 9241 210 is its insistence on addressing the whole user experience. The standard goes beyond mere functional correctness by considering how the system affects the user in terms of comfort, satisfaction, and emotional response. It requires that designers not only assess whether a system achieves its intended tasks effectively but also whether the system is acceptable from a subjective, human experience perspective. This comprehensive view of usability, incorporating factors such as cognitive load, physical comfort, and even long-term satisfaction, provides a rich framework, in which our test strategies can be contextualized. In our application, incorporating principles from ISO 9241 210 would imply designing test cases that not only measure technical metrics (execution times and error rates) but also evaluate system behaviours that influence user trust and safety.

The standard also describes the process-oriented nature of human centred design. It recommends that the design process should be iterative, where preliminary designs are refined through continuous testing, evaluation, and incorporation of user feedback. The iterative process described in ISO 9241 210 includes multiple cycles of prototype evaluation and redesign, ensuring that the final solution is well adapted to its intended context. For a testing framework, especially one aimed at HMIs in safety critical environments, this iterative methodology underscores the importance of continuously updating and refining automated tests to capture evolving user requirements and environmental constraints.

Additionally, ISO 9241 210 provides guidance on planning and managing human centred design activities. It outlines how to structure design processes, develop user requirements, and establish milestones for evaluation, which are all elements that can inform the strategic planning of our future test framework enhancements. Although our current implementation does not fully integrate all these principles, the selective adoption of insights from ISO 9241 210 enables our framework to be evaluated against robust benchmarks. In doing so, we set the stage for future work where deeper integration of these principles can lead to improved usability assessments and overall system reliability.

4.4.2 ANSI/ISA-101.01-2015

ANSI/ISA-101.01-2015 is a detailed standard focused on HMIs for systems in process automation. The standard provides precise requirements and guidelines that address the entire lifecycle of HMIs, from design and implementation through operation and continuous improvement. Unlike other broader usability standards, ANSI/ISA-101.01-2015 concentrates on how HMIs must be designed for environments where real-time monitoring and control are critical, ensuring that operators can interact safely and effectively with complex automated processes (Automation, 2015).

This standard begins by clearly defining its scope and intended audience. It targets HMI designers emphasizing that the guidance applies to any automated process control system, whether continuous, batch, or discrete. The standard reinforces that it is to be used as an aid in achieving improved reliability and quality as well as productivity in process control systems. In this way, the standard serves not only as a technical guide but also as a philosophical framework for developing robust HMIs in demanding industrial contexts.

One of the key attributes of ANSI/ISA-101.01-2015 is its focus on ensuring consistency and clarity across HMI designs. The standard regulates that HMIs must provide clear visual feedback and effective error indications. It requires that displays are designed with appropriate visual cues to enable operators to quickly discern the status of a process and to react promptly to abnormal conditions such as color, layout, and graphic symbolism. This emphasis on clarity is critical in environments where every second counts, and even minor misinterpretations can lead to significant operational hazards.

ANSI/ISA-101.01-2015 also outlines specific requirements related to the HMI's visual layout and interaction methods. Standard emphasizes that displays should be designed not only to convey information but also to support decision-making. The arrangement of screens, the use of pop-up displays for critical alerts, and the design of navigation controls are all regulated by precise guidelines aimed at reducing cognitive load and operator error. The standard requires that HMIs are developed in a way that permits rapid retrieval of information and that they incorporate redundancy and error-tolerant designs where necessary.

In our current testing framework, while we have not yet implemented all the aspects described by ANSI/ISA-101.01-2015, its detailed specifications provide valuable benchmarks. Our automated tests currently focus on verifying

functionality and responsiveness. However, this standard reminds us that future iterations should also include assessments of visual feedback, display consistency, and user interaction patterns. Practically, the improvement involves developing test cases that specifically validate the clarity of error messages, the consistency of layout elements across different screens, and the effectiveness of visual indicators under various simulated conditions.

4.4.3 Best practice in E2E testing

E2E testing account for system workflow to reflect real user behaviours and ensure high software quality. Identifying testing strategy and critical user workflows that impact business operations is an important step. Instead of aiming for fully coverage, which can result in unwieldy test cases, teams should focus on key paths where failures would cause significant disruption. Engaging cross-functional teams, with different stakeholders, helps ensure that the strategy aligns with both user needs and technical view. Prior to test scripting, clear objectives of the test can be achieved by outlining clear test scenario, listing detailed preconditions, step sequences, and expected outcomes (Margot & Mallory, 2020).

Once these essential workflows are identified, a modular test design is the next significant step. Implementing design patterns such as the Page Object Model (POM) compress complex interactions by isolating UI elements and behaviours into dedicated objects, which simplifies maintenance as the user interface evolves and enhances the readability and reusability of the test code (Dorothy, et al., n.d.). Group test cases by functionality, application modules, or user journey stages. This organization simplifies the management of an expanding test case and aids in identifying common patterns or shared dependencies. It also facilitates reusability, since tests addressing similar functionalities can often leverage identical preconditions and helper methods. Creating reusable helper functions for common tasks reduces code duplication and adjusts across multiple tests more efficient. Clear documentation through structured templates or tracking systems (Jira tickets) ensures that every test case is traceable from design through implementation. Each test case should include a brief description, list relevant preconditions, outline step-by-step actions, and specify the expected outcomes. This structured documentation not only streamlines the development phase but also aids in future maintenance and troubleshooting

In parallel, modern practices emphasize the technical environment and process elements crucial to reliable testing. Containerized testing environments, such as running tests within Docker containers on dedicated servers, guarantee that the test environment remains reproducible and isolated. It

significantly reduces setup time and minimizing environmental drift. This controlled isolation has been validated by multiple studies as a key contributor to consistency across test runs (Rex, et al., 2009). Furthermore, integrating the testing pipeline with version control systems and continuous deployment processes creates a robust feedback loop, where continuous integration (CI) is leveraged to trigger automated test execution on every commit through CI/CD pipelines, can enable rapid feedback and early detection of defects, which ultimately reduces the overall development cycle and associated costs (Chuanzan, et al., 2020).

Robust reporting is another essential aspect. The detailed and interactive HTML reports that include screenshots, trace information, and comprehensive logs allow developers to quickly point out the root causes of failures. Structured reports, formatted as JUnit or JSON, can friendly integrate with CI systems, supporting automated build status reporting and facilitating data-driven decision-making. Besides that, synchronization mechanisms, such as native auto-waiting and explicit wait constructs, ensure that tests only proceed once all necessary UI components have fully loaded, thereby minimizing flaky tests which is the pain point of E2E tests.

Beyond technical aspects, long-term success in E2E testing requires adaption to best practices that enhance maintainability and scalability. Tests should be organized into logical sets by functionality or user journey stages, and stable selector strategies should be implemented using attributes like unique “data-testid” markers or element roles that remain consistent even if the underlying HTML changes. Employing fixtures to manage setup and teardown processes guarantees that tests run in isolation by always beginning from a known, controlled state (Playwright, n.d.). Additionally, a comprehensive test strategy, designed with input from cross-functional teams such as product managers, QA experts, and developers, is important to ensure that testing aligns with both user needs and technical feasibility (Golubeva, 2024).

4.4.4 Current alignment with Standard and best practice

Criterion	Current Implementation	Future Enhancements
User Focus (ISO 9241-210)	Test cases simulate critical plant operators and system engineer workflows	Recursive stakeholder feedback loops to keep test scenarios in sync with rapid HMI changes
Iterative Process (ISO 9241-210)	CI/CD runs tests on every commit.	Scheduled, data-driven user reviews to adjust tests based on trends of data and feedback
Comprehensive Usability (ISO 9241-210)	Tests cover technical metrics (speed, errors), ensuring the application performs as expected under normal conditions	Add visual-feedback checks: assert event feedback time, error clarity & UI presentation tests
Safety Criticality (ISO 9241-210)	Test cases include most critical workflows	Broader stress-testing and failure-mode simulations to validate safety under extreme conditions and abnormal situation
Visual Clarity (ANSI/ISA-101.01-2015)	Assertions verify text, elements & responsiveness	Inject Axe for color-contrast audits, snapshot comparisons & viewport rotations
Operational Robustness (ANSI/ISA-101.01-2015)	Validates core workflows and error paths	Simulate real-world operational stresses, faults and recovery mechanisms
Reproducible Environment	Tests run in Docker on Ubuntu ensuring consistent environments and reducing setup overhead	Integrate cAdvisor/Prometheus/Grafana for monitoring Multi-stage Dockerfiles and automated image-size budgeting
Continuous Integration	Bamboo pipeline triggers tests, reports & gates PRs	Real-time Grafana dashboards for build/test metrics; SonarQube quality gates in CI
Comprehensive Reporting	Playwright HTML, JUnit & JSON reports; no trend data	Add visual analytics, long-term trend aggregation, video/storage solution with Allure or similar
Synchronized Testing	Uses Playwright's built-in waits	Adaptive wait wrappers based on historical load times
Test Modularity & Feedback	POM structure; results linked to Bitbucket commits/PRs and Slack stakeholder notification channel	Further decompose tests, add metadata tagging, integrate Zephyr for grouping & traceability
Test Documentation & Traceability	Structured templates; no automated traceability matrix	Connect to Jira/Xray or Jama, auto-generate traceability in Confluence, embed templates
Test Data Management	Docker baseline; limited seeding/cleanup automation	Scripts for data init/refresh + cleanup; archive test data for debugging and trend analysis
Comprehensive Test Strategy	Ad-hoc workflows based on stakeholder consultants	Formal strategy doc in Confluence; integrate with Jira; periodic cross-team reviews
Test Maintainability	Modular design & helpers; informal conventions	Enforce ESLint/Prettier, SonarQube for debt analysis; schedule regular refactoring sprints

Our E2E suite is built to simulate practical operator workflows and environmental conditions in the HMI, ensuring that each scenario mirrors real-world usage and validates critical functionality. We begin by automating the core power-plant

operator tasks, from navigating screens, responding to the alarms system, and applying configuration profiles via internal APIs, and extending coverage to the complex configuration variants used across different plant types. These “horizontal” test cases were defined in close collaboration with HMI engineers and system operators to surface the highest-priority user journeys and business requirements early in development.

Rather than simple pass/fail markers, our tests include rich assertions that verify text and layout consistency, confirm event-driven behaviors, and exercise data-driven workflows end to end. To eliminate the flakiness that is pain point of many UI E2E test, we rely on Playwright’s built-in auto-waiting and supplement it with targeted explicit waits, guaranteeing that every dynamic component is fully loaded before interaction.

We structure our tests in a modular Page Object Model, with each test functioning as an independent unit and helper functions abstracting common interactions. This design lets us add new scenarios without touching existing code, keeping the codebase clean and easy to maintain as the HMI’s complexity grows.

Our test results are constructed to generate interactive HTML reports (with screenshots, steps, and trace captures) along with structured JUnit and JSON reports that friendly integrate with our CI/CD system. These test reports provide immediate material for developers to identify the issue caused by the new change in HMI code. This instantaneous is also emphasized by our CICD pipeline design, where the reports are propagated to developers through multiple channels.

When a test run completes, Playwright generates an interactive HTML report, complete with embedded steps, screenshots, and trace captures, as well as structured JUnit XML and JSON artifacts that friendly integrate with our CI/CD system. These outputs are automatically archived and linked into our CI/CD pipeline, giving developers immediate visibility into failures and the precise context needed to diagnose issues.

Our CI/CD orchestration, managed by Bamboo, triggers this entire process on every commit. Pull requests are gated on a clean test pass. Any failure blocks the merge and instantly alerts the team via linked PR comments and Slack notifications. We also capture execution times and error rates as metrics, ensuring that core workflows remain performant and reliable under normal operating conditions.

Finally, every test runs in a isolated Docker container on our Ubuntu test servers. By spinning each test case up from a predefined image and tearing it down afterward, we eliminate environment drift, minimize setup overhead, and keep our feedback loop both fast.

4.5 Summary

Building on the testing framework requirements we defined with stakeholders in Chapter 3, this chapter discusses our tools selection and architecture decision.

We evaluated five leading E2E tools Cypress, Selenium, Puppeteer, Nightwatch, and Playwright against must-have features like running custom JavaScript, supporting our custom Chromium build, simulating clicks reliably, waiting for dynamic content, and fitting into our CI/CD flow. Playwright turned out to be the best selection thanks to its built-in auto-waiting, cross-browser coverage, and easy tracing.

With the selected tools and the essential requirement for testing framework we defined with stakeholders, we sketch our architecture: every Bitbucket push fires a Bamboo job that checks out the exact commit, spins up Docker containers pre-loaded with Node.js, Playwright and browser engines, runs tests in parallel agents, then pushes interactive HTML reports, JUnit XML, JSON artifacts and concise console logs back into the pull request and notify Slack channel. Our report and test run output is carefully defined with formats and performance metrics, so developers get rapid, actionable insights into regressions and system health.

5 Implementation

5.1 Server

All of our Playwright agents run on the same Ubuntu VM under Bamboo's Docker runner. That host specs:

- 2-vCPU,
- 4 GB-RAM Intel Xeon Platinum 8259CL instance
- KVM virtualized.

We currently run up to four parallel containers (one per Bamboo plan) against that single host. Under full load, our metrics show CPU spikes up to 80 percent and memory usage topping out around 3 GB across all agents. Disk I/O stays low, since most artifacts are written over the network to Bamboo's shared storage.

5.2 Implementation

We implemented two core end-to-end workflows to validate HMI functionality: a pagination test that navigates a data table and confirms both the UI update and underlying API response, and a setpoint test that opens a modal dialog, submits a new value, and verifies the change in the interface and configuration API. Each step is wrapped in Playwright's `test.step()`, generating clear action–verification pairs in our HTML reports and capturing network traces for deeper assertion.

Over a two-week window encompassing 45 commit-triggered runs, we collected timing and stability metrics. Let T_i be the total duration of run i . The mean execution time

$$T = \frac{1}{45} \sum_{i=1}^{45} T_i = 275 \text{ seconds}$$

with a minimum of 240 seconds and a maximum of 350 seconds. The average durations for the individual scenarios were 10 seconds (pagination) and 20 seconds (setpoint). In early deployments we observed longer test run total duration due to a cold-start penalty the first time the original Docker image was pulled and built up, which took up to 350 seconds in total. This means that in our current setup, nearly eighty percent of the total runtime is consumed by environmental overhead. To drive that overhead down, we are experiencing layering browser binaries and core dependencies into base image so that only application code changes invalidate the Docker cache. Multi-stage builds further isolate infrequently changing components, like operating

system packages, from test code, meaning rebuilds only recompile what is necessary. In parallel, we are also trying a pool of always-running containers orchestrated via Kubernetes, which would eliminate image-pull and setup time altogether and deliver consistently fast feedback on every commit.

We quantify flakiness as the proportion of tests that only pass after a retry. Formally:

$$\textit{Flakiness Rate (FR)} = \frac{N_{\text{retry-pass}}}{N_{\text{total-runs}}} \times 100\%$$

where $N_{\text{retry-pass}}$ is the number of test executions that failed on the first attempt but succeeded on retry, and $N_{\text{total-runs}}$ is the total number of test executions.

Over our last 45 test runs we saw 2 cases that needed a second retry, so

$$FR = \frac{2}{45} \times 100\% = 4.44\%$$

yielding a flakiness rate of about 4.44 percent.

This small flakiness rate reflects the effectiveness of Playwright's built-in auto-waiting and our targeted explicit wait strategies. Allowing a single retry raised our overall pass rate to one hundred percent.

Besides performance, our suite has significantly enhanced code coverage and developer productivity. Developers now receive test results in under five minutes after pushing changes, eliminating the median pull-request turnaround from hours to 10 minutes and saving the team hours of manual testing effort every day.

Even further the test result feedback quality, our Playwright test suite produces three coordinated report artifacts on every run. An interactive HTML report, a JUnit-format XML, and a JSON results file, that collectively surface both high-level outcomes and low-level diagnostics. The HTML report offers a navigable view of each test step, complete with embedded screenshots and trace snapshots for any failed assertions. The JSON report provides a machine-readable log of every test's metadata, that we can later parse for trend analysis or integrate into external dashboards. As soon as Playwright finishes, our Bamboo plan archives all three artifacts and generates a unique build URL. That URL is then injected into a comment on the

corresponding Bitbucket so that every developer sees a “View E2E report” link directly under their code changes. This linkage ensures that anyone reviewing the PR has immediate, one-click access to both the high-level result and the full diagnostic detail without leaving their code review workflow.

In practice, we achieved a consistent one hundred percent gating of pull requests, no failing tests ever slipped through testing to production, while keeping test executing duration modest. Even though our current coverage spans only a small percentage of critical UI workflows, the modular infrastructure and centralized selector strategy allow us to onboard new scenarios rapidly. With just a few lines of configuration, we can now add tests for alarm-dialog sequencing, or any newly planned operator work flow.

Beyond these quantitative milestones, the implementation process deepened our understanding of the HMI’s behaviour, uncovering undocumented API parameters, race conditions in rapid setpoint updates, which have already seeded new test scenarios and performance optimizations. This implementation and the knowledge provide the foundation for the next development iteration.

Resource monitoring on our 2 vCPU, 4 GB-RAM Ubuntu VM running four parallel Docker agents showed CPU utilization peaking at 80 percent and memory usage holding around 3 GB. Disk I/O remained minimal, as artifacts streamed directly to Bamboo’s shared storage. These measures confirm that our existing infrastructure easily supports the current workload. However, projecting a 50 percent increase in test coverage, by adding workflows, would push CPU utilization well beyond comfortable limits, so horizontal scaling via Kubernetes or additional build agents to preserve our fast feedback loop is needed.

In summary, this first implementation phase has not only validated our architectural choices but also delivered a robust framework that meets the project’s goals with high effectiveness. At the same time, it establishes a clear roadmap for future enhancements.

6 Discussion

Our automated testing framework has demonstrated that a requirements-driven approach, defined in close stakeholder collaboration, can have a reliable end-to-end solution for safety-critical HMIs. By analysing the essential needs such as custom JavaScript execution, Chromium compatibility, reliable click simulation, dynamic-content waits, content verification, and CI/CD integration, we ensured that our solution design directly address the practical challenges faced by power-plant operators and HMI design engineers.

From an architectural point of view, containerizing each commit-triggered test run in Docker under Bamboo not only eliminated environment drift but also delivered reproducible and rapid feedback solution. The average end-to-end execution time of 275 seconds can provide developers with immediate insight into performance regressions. The measured flakiness rate of 4.4 percent, which dropped to zero with one retry, underscores the robustness of Playwright's synchronization mechanisms and our targeted explicit waits. These metrics show that the framework meets the demands of speed and stability required in a continuous-integration workflow.

At the same time, our evaluation confirms that our current setup, running up to four parallel containers on a 2-vCPU, 4 GB-RAM VM, comfortably supports our two-scenario suite without overloading the host (peak CPU at 80 percent, memory at 3 GB). Although four parallel containers performed acceptably for our two scenarios, we cannot claim that a single host will suffice as the number or complexity of tests increases. However, when we expand to more tests or more complex workflows, we will have to either provision a more powerful server or optimize the current architecture and introduce elastic agent scaling to maintain performance.

Our reporting strategy, interactive HTML with trace captures, JUnit XML, and JSON artifacts, providing insight for effectively identifying failures. Developers could immediately navigate from a pull request to a detailed step-by-step narrative, greatly reducing mean time to resolution. However, capturing only isolated snapshots limits our ability to analyse performance drift or flakiness trends across longer periods. Integrating a historical analytics platform such as Grafana or Allure would enrich our insight, enabling data-driven decisions about when to refactor tests or adjust timeouts.

Finally, aligning the framework with ISO 9241-210 and ANSI/ISA-101.01 principles validated our human-centred and safety-focused design choices while revealing areas for growth. Our tests simulate core operator workflows,

but they do not yet measure cognitive load or verify visual clarity under stress conditions. Incorporating axe-core accessibility audits and viewport-based pixel-diff checks would extend our coverage into user usability. Moreover, formalizing traceability through metadata-driven Page Objects and Zephyr integration would close the loop between requirements, test code, and failure artifacts.

Importantly, by fully automating end-to-end validation of critical operator workflows, our framework has effectively replaced what was once a labour-intensive, error-prone manual testing process. Developers now receive consistent, reproducible feedback on every commit, freeing QA resources to focus on exploratory and usability testing rather than repetitive regression checks.

Our work answers the research questions by laying out clear requirements, demonstrating a Docker-Bamboo-Playwright architecture satisfying those needs, structuring metric outputs into actionable reports, and showing how international standards can guide future enhancements. The framework delivers continuous feedback on every commit, significantly reducing manual effort and risk. At the same time, the discussion addresses key limitations—single-host scaling, evolving usability checks, and long-term trend analysis—that would be the direction of the next phase of development.

7 Conclusion

This study goal is to identify the essential requirements for automated HMI testing, and accordingly propose an automated testing framework architecture, which meets the requirement of test outputs, and align with best practices as well as international standards.

Stakeholder workshops defined a set of technical needs: the ability to execute custom JavaScript for internal API calls in browser context, compatibility with our customized Chromium executable, robust click simulation, reliable waiting for dynamic content, comprehensive content and translation verification, and tight CI/CD integration.

Moreover, we demonstrated that our Docker-Bamboo-Playwright architecture best meets these requirements. Docker containers guarantee reproducible environments on every commit. Bamboo orchestrates provisioning, test execution, artifact collection, and reporting. Playwright delivers native auto-waiting and trace capture that minimize flakiness and speed test development respectively.

Our reporting outputs, interactive HTML with embedded screenshots and traces, JUnit XML, and JSON artifacts, providing developers with instant, insight into regression failures and performance metrics. These formats friendly integrate with pull-request workflows and notification channels, collapsing the feedback loop to minutes rather than hours.

Importantly, by benchmarking our design against field best practice, ISO 9241-210's human-centred principles and ANSI/ISA-101.01's clarity and safety guidelines, we validated our focus on operator workflows and identified future enhancements such as accessibility audits, pixel-diff visual checks, and metadata-driven traceability.

In summary, our work answers the core research questions and delivers a continuous, production-grade feedback mechanism for our safety-critical HMI development process. It replaces error-prone manual regression with an automated, reliable process that scales with development velocity. At the same time, the limitations we have identified will guide future research into elastic scaling, extended scenario coverage, long-term trend and historical data analysis, deeper integration with industrial standards.

8 Future work

Although our current framework delivers reliable, production-grade feedback, there remain several opportunities to strengthen both its effectiveness.

First, we will organize recursive stakeholder feedback loops by scheduling regular review sessions with system engineers and development team. Capturing emerging workflows and pain points in real time will allow us to refine and add test scenarios so that our suite stays synchronized with rapid HMI development.

Second, we plan to introduce adaptive, data-driven waiting strategies and deep accessibility audits. During every run we will log actual selector resolution times and calculate dynamic timeouts as the mean plus two standard deviations. When a component exceeds its historical timing profile the test will fall back on a safe maximum, minimizing flakiness and delays. In parallel we will inject axe-core contrast rules, capture full-page screenshots across multiple viewports, and perform pixel-diff comparisons against approved baselines. This will enable early detection of regressions in color contrast and layout consistency.

Our Page Object Model will evolve into a metadata-driven architecture in which each atomic module is tagged by feature area and user story. We will integrate these tags with a test-management system such as Zephyr for Jira, linking code, test definitions, and requirements in a single traceability matrix.

Regarding reports, we will transition from standalone HTML snapshots to a historical analytics platform. By streaming test outcomes, performance metrics, and artifact links into Prometheus and Grafana or deploying an Allure server, we can build dashboards that surface creeping flakiness, turning each test run into an actionable event. We also plan to enrich our Slack and Teams notifications with contextual alerts that include metrics trends and links to relevant dashboards.

Finally, we will optimize our Docker images and test infrastructure. By running cAdvisor alongside each container we can collect fine-grained telemetry. Multi-stage builds will enforce image-size budgets, while container-native orchestration, possibly leveraging Kubernetes, will allow elastic scaling of test agents based on real-time load.

Together, these enhancements will ensure that our framework not only keeps pace with evolving HMI requirements but continues to deliver fast, meaningful feedback.

References

Anon., 2020. *testim*. [Online]
Available at: <https://www.testim.io/blog/end-to-end-testing-best-practices/>
[Accessed April 2025].

Anon., 2021. *ibm*. [Online]
Available at: <https://www.ibm.com/think/topics/software-testing>
[Accessed March 2025].

Automation, I. S. o., 2015. *Human Machine Interfaces for Process Automation Systems - ANSI/ISA-101.01-2015*. s.l.:s.n.

Beata, B. & Agnieszka, A., 2024. Analysis of end-to-end test automation tools based on the examples of Selenium WebDriver and Playwright. *19th Conference on Computer Science and Intelligence Systems*.

Chuanzan, W. et al., 2020. Human-Machine Interaction in Future Nuclear Power Plant Control Rooms – A Review. *IFAC-PapersOnLine*, pp. 851-856.

Devaraj, K., 2025. *testsigma*. [Online]
Available at: <https://testsigma.com/blog/test-suite-vs-test-case/>
[Accessed April 2025].

Dorothy, G., Erik, v. V., Isabel, E. & Rex, B., n.d. FOUNDATIONS OF SOFTWARE.

Eduard, E., Daniel, S., Adnan, Č. & Paul, P., 2017. A Comparative Study of Manual and Automated Testing for Industrial Control Software. *2017 IEEE International Conference on Software Testing, Verification and Validation (ICST)*.

Eduard, P. E., Daniel, S., Adnan, C. & Paul, P., 2017. A Comparative Study of Manual and Automated Testing for Industrial Control Software. *Conference: 2017 IEEE International Conference on Software Testing, Verification and Validation (ICST)*.

Golubeva, M., 2024. *medium*. [Online]
Available at: <https://medium.com/jagaad/14-test-automation-best->

practices-ffd5688of2oe
[Accessed March 2025].

Himanshu, K., n.d. *marutitech*. [Online]
Available at: https://marutitech.com/qa-in-cicd-pipeline/?utm_source=chatgpt.com
[Accessed 30 April 2025].

ISO, 2019. *Ergonomics of human-system interaction - ISO 9241-210:2019*. s.l.:s.n.

Iva, K., Gebremariam, A., Onur, D. & Andrea, A., 2025. A Survey on Web Testing: On the Rise of AI and Applications in Industry.

Jetbrains, n.d. *jetbrains*. [Online]
Available at: <https://www.jetbrains.com/teamcity/ci-cd-guide/concepts/flaky-tests/#:~:text=Flaky%20tests%20are%20defined%20as,code%20or%20the%20test%20itself>
[Accessed April 2025].

Margot, L. & Mallory, M., 2020. *datadoghq*. [Online]
Available at: <https://www.datadoghq.com/blog/test-creation-best-practices/>
[Accessed April 2025].

Maurizio, L., Davide, P. & Filippo, R., n.d. *Mutta: a novel tool for E2E web mutation testing*. s.l., s.n., p. 5–26.

Maurizio, L., Diego, C., Filippo, R. & Paolo, T., 2016. Approaches and Tools for Automated End-to-End Web Testing. In: *Advances in Computers*. s.l.:s.n., p. 193–237.

Peter, S. S. & Katalina, G., 2015. Manual to Automated Testing: An Effort-Based Approach for Determining the Priority of Software Test Automation.
Pittet, S., n.d. *atlassian*. [Online]
Available at: <https://www.atlassian.com/continuous-delivery/software-testing/types-of-software-testing>
[Accessed April 2025].

Playwright, n.d. *playwright*. [Online]
Available at: <https://playwright.dev/docs/test-fixtures>
[Accessed March 2025].

Rex, B., Erik, v. V. & Dorothy, G., 2009. *Foundations of Software Testing*.
s.l.:Cengage Learning EMEA.

Ruari, M., n.d. *marketscale*. [Online]
Available at: <https://marketscale.com/industries/engineering-and-construction/whats-the-true-cost-of-machine-downtime/>
[Accessed April 2025].

Rudolf, R., Werner, P. & Dietmar, W., 2014. Automated testing of
industrial automation software: Practical receipts and lessons learned.

Saad, A., Samia, A. & Tahani, A., 2025. Comparative Reliability Analysis
of Selenium and Playwright: Evaluating Automated Software Testing Tools.
Asian Journal of Research in Computer Science.

Schmitt, J., 2024. *circleci*. [Online]
Available at: <https://circleci.com/blog/testing-pyramid/>
[Accessed March 2025].

Vøhid, G. & Mika, M., 2016. When and what to automate in software
testing? A multi-vocal literature review. *Information and Software
Technology*.

Wang, C. et al., 2020. Human-Machine Interaction in Future Nuclear
Power Plant Control Rooms – A Review. *IFAC-PapersOnLine*, pp. 851-856.

Appendix

Appendix A: Playwright test runner configuration (playwright.config.js)

```
1. module.exports = defineConfig({
2.   workers: 4,
3.   testDir: './src/tests',
4.   reporter: [
5.     ['html', { open: 'never', outputFolder: 'playwright-report' }],
6.     ['junit', { outputFile: 'playwright-report/junit/results.xml' }],
7.     ['json', { outputFile: 'playwright-report/json/results.json' }],
8.     ['line'],
9.   ],
10.  grep: new RegExp(testCases),
11.  retries: 1,
12.  use: {
13.    browserName: 'chromium',
14.    channel: 'chrome',
15.    headless: true,
16.    viewport: { width: 1280, height: 720 },
17.    ignoreHTTPSErrors: true,
18.    trace: 'retain-on-failure',
22.    launchOptions: {
23.      executablePath: process.env.PLAYWRIGHT_BROWSERS_PATH || undefined,
24.    }
25.  },
26. });
27.
```

Appendix B: AWS SSH provisioning and known hosts configuration

```
1. mkdir -p ~/.ssh
2. echo -n $AWS_SSH_KEY | base64 -d > ~/.ssh/aws_key
3. chmod 600 ~/.ssh/aws_key
4. ssh-keyscan -H <server-ip> >> ~/.ssh/known_hosts
5.
```

Appendix C: Port HMI E2E test report to Bitbucket Pull Request

```
1. TEST_REPORT_URL="http://bamboo-server/reports/$(cat build.properties)"
2. curl -X POST -H "Content-Type: application/json" \
3.   -d '{"text": "HMI E2E Test Results: Review the detailed report at
${TEST_REPORT_URL}"}' \
4. https://api.bitbucket.org/2.0/repositories/repo/pullrequests/${PR_ID}/comments
5.
```

```
1. RUN apt-get update && \
2.   apt-get install -y git tmux build-essential bash && \
3.   rm -rf /var/lib/apt/lists/*
4.
```

Appendix D: Repository clone and branch synchronization procedure

```
1. echo "[Step 1] Cloning repository..."
2. git clone ssh://git@SSH-GIT-HOST:<PORT>/<org>/<repo-name>.git
3. cd <repo-name>
4. git checkout <branch-name>
5. git pull origin <branch-name>
6.
```